Vehicle Tracking Device

Bachelor of Technology Project Progress Report

Sagar Baruah and Ponting
IIT Roorkee
24 October 2026

1 Progress summary

We have integrated an ADXL345 accelerometer with the Heltec CubeCell AB02S prototype and obtained live acceleration readings on the onboard OLED and Serial Monitor. A laptop visualization demonstrates the measured acceleration vector relative to reference axes. These results establish the sensor acquisition stage of the vehicle tracking device.

Earlier project work established a vehicle tracking node and a basic tracking dashboard. The next stage is to validate GPS speed in km/h, combine GPS and acceleration acquisition, and transmit the combined data to the fleet dashboard. Vehicle trials and accident detection remain future work.

2 Project objective

The project targets campus e-rickshaw fleet visibility. Each vehicle node is intended to report its location, speed and motion data so that a central dashboard can show vehicle movement and support later studies of route coverage and vehicle availability.

3 Current implementation status

Subsystem	Progress
Acceleration acquisition	Working ADXL345 register reads with OLED and Serial output
Acceleration visualization	Laptop demonstration available
Tracking dashboard	Basic dashboard reported in earlier project work
GPS speed	Initial standalone sketch prepared; field validation pending
Combined telemetry	GPS, acceleration and dashboard integration pending

4 Prototype hardware

The current bench prototype connects the ADXL345 breakout to the CubeCell board over I2C. The onboard OLED provides immediate feedback during sensor bring-up. The breadboard arrangement supports wiring changes before a more secure vehicle installation.

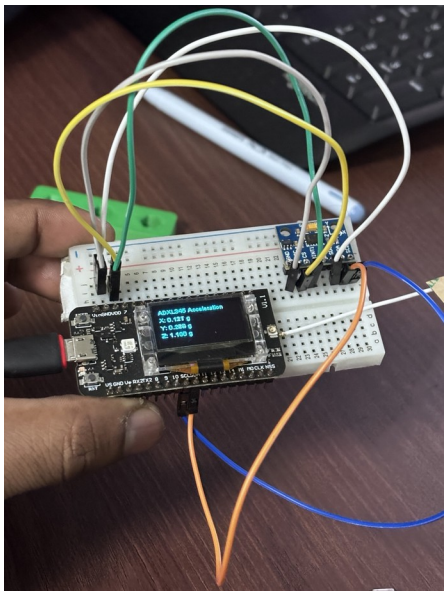


Figure 1 Working breadboard prototype showing acceleration on the OLED

5 Firmware implementation

The firmware enables Vext, initializes the OLED and searches the two ADXL345 I2C addresses, 0x53 and 0x1D. It accepts the device only when the identification register returns 0xE5. It then configures full-resolution measurement and starts reading the three axes.

Setting	Value in the working sketch
I2C bus	100 kHz OLED configuration on SDA and SCL
Measurement configuration	Full resolution with a ±2 g range
Sensor output data rate	25 Hz, configured using register 0x2C = 0x08
Acquisition and display interval	100 ms, approximately 10 readings per second
Data conversion	Six bytes from 0x32; signed 16-bit values scaled by 0.0039 g

If initialization fails, the firmware retries and shows a wiring message. If a later read fails, it reports a read error rather than printing uninitialized axis values.

6 Demonstration and observations

The prototype photo confirms that the OLED displays X, Y and Z acceleration in g. The supplied demonstration video shows an acceleration vector with reference and sensor-axis labels on a laptop. This gives a practical visual check of axis response during handling.

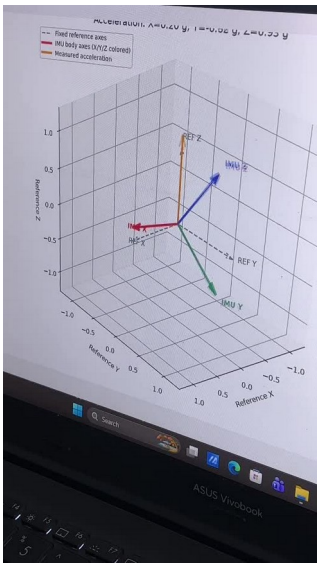


Figure 2 Frame from the acceleration visualization demonstration

These demonstrations confirm basic acquisition and visualization. They do not establish sensor accuracy, vehicle performance or reliable accident detection. At rest, the sensor includes gravity, so an axis aligned vertically can read close to 1 g. Mounting angle must therefore be considered when interpreting vehicle acceleration.

7 GPS speed tracking

The next implementation reads speed directly from the onboard GPS and displays it in km/h. The initial standalone sketch also displays satellite status and suppresses speed when GPS location or speed data become stale. It assumes an AIR530Z GPS version and requires verification against the installed board and library.

Outdoor testing will first confirm a stable GPS fix. Stationary tests will record speed drift, followed by moving tests against a reference speed source. GPS parsing must run continuously when the accelerometer and OLED are added to the same firmware. No measured GPS speed accuracy is claimed at this stage.

8 Current limitations

The present sketch reads the sensor about 10 times per second even though the sensor output rate is 25 Hz. Fast motion events may be missed, and larger shocks may exceed the selected ±2 g range. The ADXL345 measures acceleration only; it does not provide a gyroscope or a complete orientation estimate. Calibration, secure mounting and vehicle vibration tests are still required.

9 Telemetry and dashboard integration

The intended data path links the vehicle node to a LoRa telemetry receiver and then to timestamped backend records and the fleet dashboard. The current accelerometer sketch operates locally and does not yet transmit combined sensor packets.

The planned packet contains vehicle ID, timestamp, latitude, longitude, GPS speed, three acceleration axes and a GPS validity flag. Including validity and timestamps allows the dashboard to distinguish a current reading from a stale or unavailable location. Packet encoding and transmission intervals will be selected during integration.

LoRa transport and any proposed custom mesh require separate protocol and coverage testing. A working sensor or radio link alone does not establish fleet-wide routing capability.

10 Next milestones and validation

Milestone	Validation activity
GPS speed validation	Obtain outdoor fixes; record stationary drift; compare speed during movement
Combined firmware	Run continuous GPS parsing with independent sensor and display schedules
Sensor calibration	Record stationary bias and controlled axis orientations; choose suitable range and rate
Combined transmission	Verify decoded packet fields, timestamps and GPS validity at the receiver
Dashboard integration	Show position and speed; indicate stale data and missing updates
Vehicle trials	Measure packet delivery, coverage and power use on representative campus routes

11 Conclusion

The current milestone delivers working acceleration acquisition, local display and a visual demonstration. The immediate priority is GPS speed validation followed by combined telemetry. These steps will connect the bench prototype to the broader campus fleet tracking system and provide the measurements needed to assess later motion-event features.

Evidence record

Working Arduino sketch provided with this progress update; prototype photograph IMG_0092.jpeg; acceleration demonstration IMG_0093.mp4. The earlier project discussion records the tracking-node and basic-dashboard baseline. Field accuracy, radio coverage and battery-life results have not yet been established.

Appendix A Sensor initialization code

The following excerpts come from the working Arduino sketch. The register-write helper returns false if the I2C transaction fails. The complete sketch also searches addresses 0x53 and 0x1D and verifies device ID 0xE5 before configuration.

Register write helper

```
bool writeRegister(uint8_t reg, uint8_t value)
{
  Wire.beginTransmission(sensorAddress);
  Wire.write(reg);
  Wire.write(value);

  return Wire.endTransmission() == 0;
}
```

Measurement configuration

Excerpt from beginADXL345 after successful address and device identification. The function exits on a failed register write.

```
  // Enter standby mode.
  if (!writeRegister(0x2D, 0x00)) return false;

  // Full-resolution mode, +/-2 g range.
  if (!writeRegister(0x31, 0x08)) return false;

  // Output data rate: 25 Hz.
  if (!writeRegister(0x2C, 0x08)) return false;

  // Start measurement.
  if (!writeRegister(0x2D, 0x08)) return false;

  delay(100);
  return true;
```

Power control

```
void powerOn()
{
  pinMode(Vext, OUTPUT);
  digitalWrite(Vext, LOW);
  delay(100);
}
```

The OLED constructor in the sketch selects 0x3C, a 100 kHz bus, SDA and SCL, a 128 by 64 display and GPIO10. setup initializes the display and calls Wire.begin before sensor detection.

Appendix B Acceleration acquisition code

The function reads all six axis bytes starting at DATAX0, combines each little-endian pair into a signed value, and converts the results to acceleration in g.

```
bool readAcceleration(float &x, float &y, float &z)
{
  uint8_t data[6];

  if (!readRegisters(0x32, data, 6)) {
    return false;
  }

  int16_t rawX = (int16_t)(
    ((uint16_t)data[1] << 8) | data[0]
  );
  int16_t rawY = (int16_t)(
    ((uint16_t)data[3] << 8) | data[2]
  );
  int16_t rawZ = (int16_t)(
    ((uint16_t)data[5] << 8) | data[4]
  );

  x = rawX * 0.0039f;
  y = rawY * 0.0039f;
  z = rawZ * 0.0039f;

  return true;
}
```

Reading interval and failure handling

Excerpt from loop. The full sketch also displays the failure message on the OLED and prints or displays each valid X, Y and Z reading.

```
  // Refresh readings every 100 ms.
  if (millis() - lastReadAt < 100) {
    return;
  }

  lastReadAt = millis();
  float x, y, z;

  if (!readAcceleration(x, y, z)) {
    Serial.println("ADXL345 read failed");
    display.clear();
    display.drawString(0, 0, "ADXL345 READ ERROR");
    display.drawString(0, 16, "Check connections");
    display.display();
    return;
  }
```

These are report excerpts, not a standalone sketch. They use the original global declarations, Wire initialization, display object and readRegisters helper. GPS speed integration remains a separate next milestone.